Computer Science

14. Docker, VM, Cloud

14주차: 도커, VM, 클라우드

목차 INDEX

01 컨테이너와 도커

- Deployment
- Virtual Machine
- Container
- Docker

02 클라우드 컴퓨팅

- Cloud Computing
- IaaS
- PaaS
- SaaS

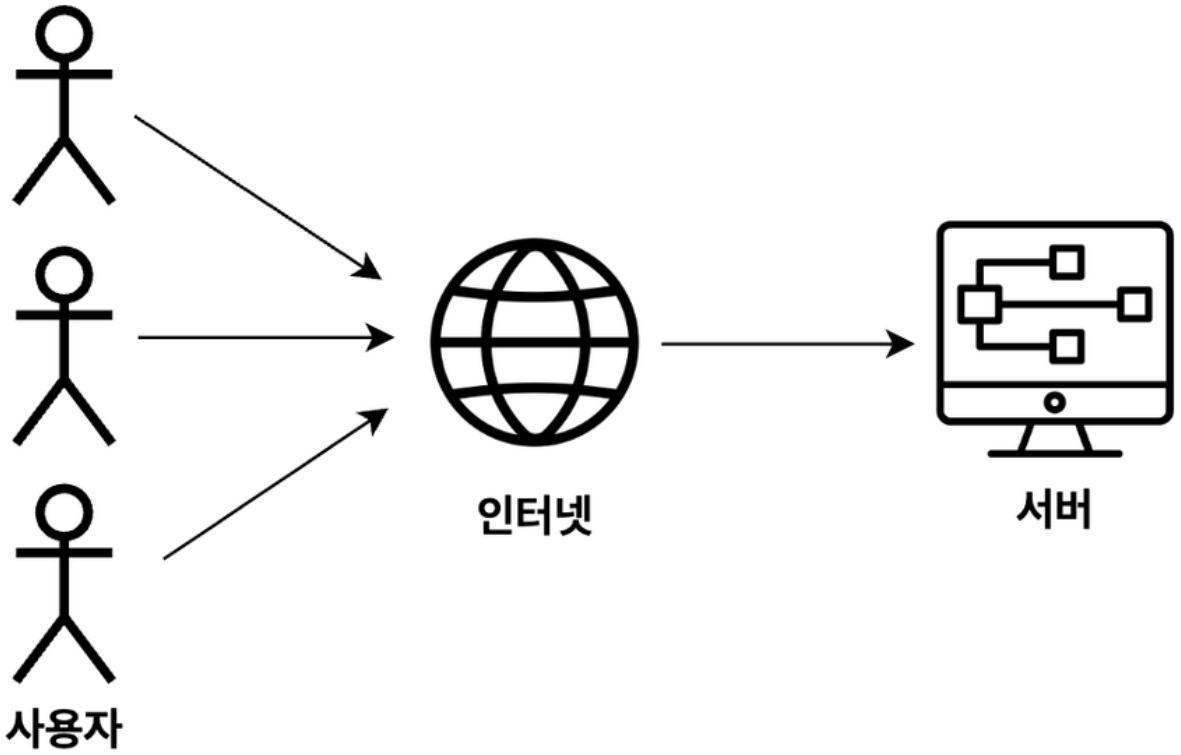
03 실무 맛보기

- Refactory
- NGINX
- MQTT & AMQP
- Kafka
- Monitoring
- Kubernetes
- ELK
- CI/CD
- DevOps

01
배포와 가상화

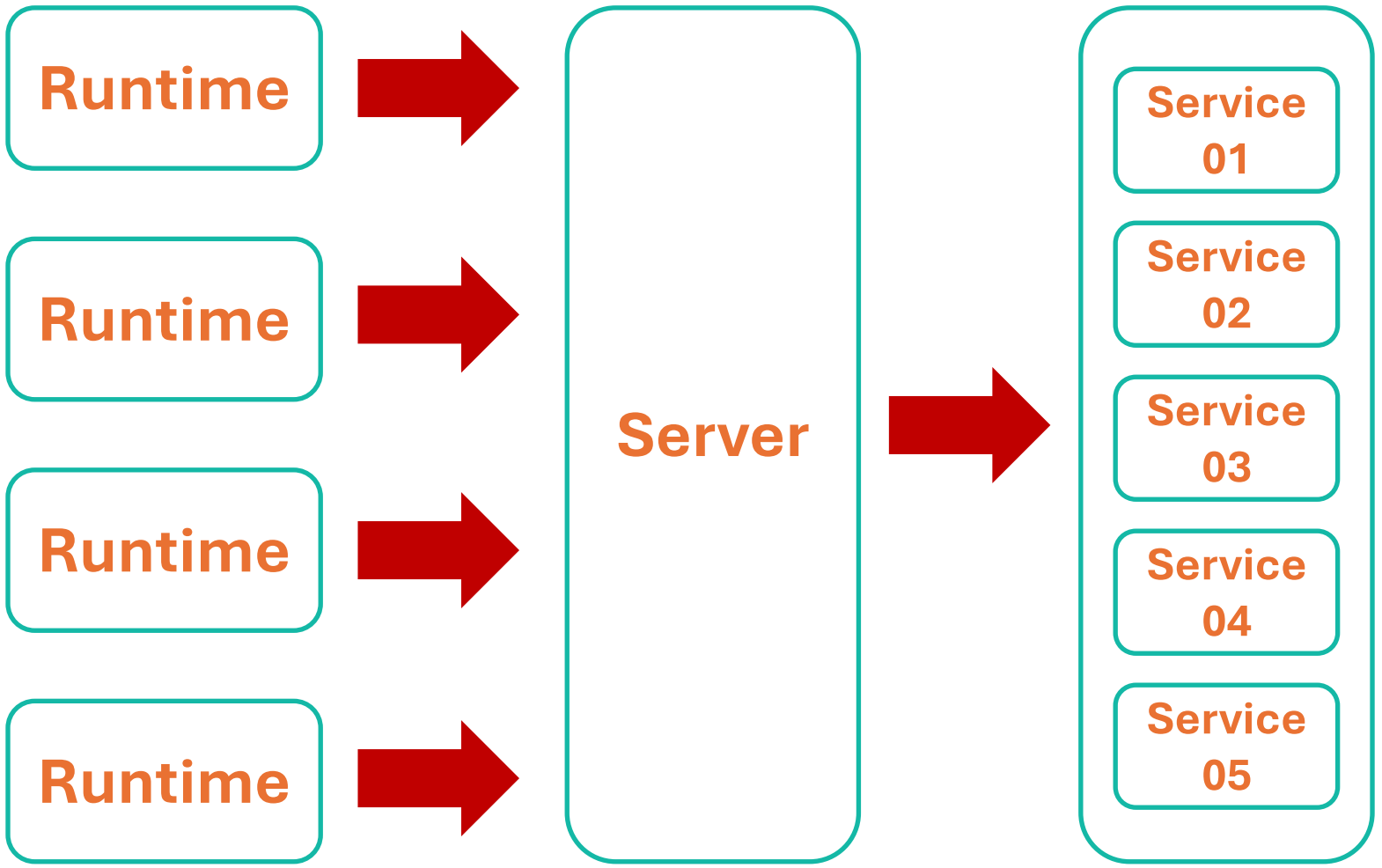
Deploy and Virtualization

Deployment (배포)



구현이 끝난 서비스를 실제로 인터넷에 올려
사람들에게 배포하는 것

배포 서버 최적화하기: VM



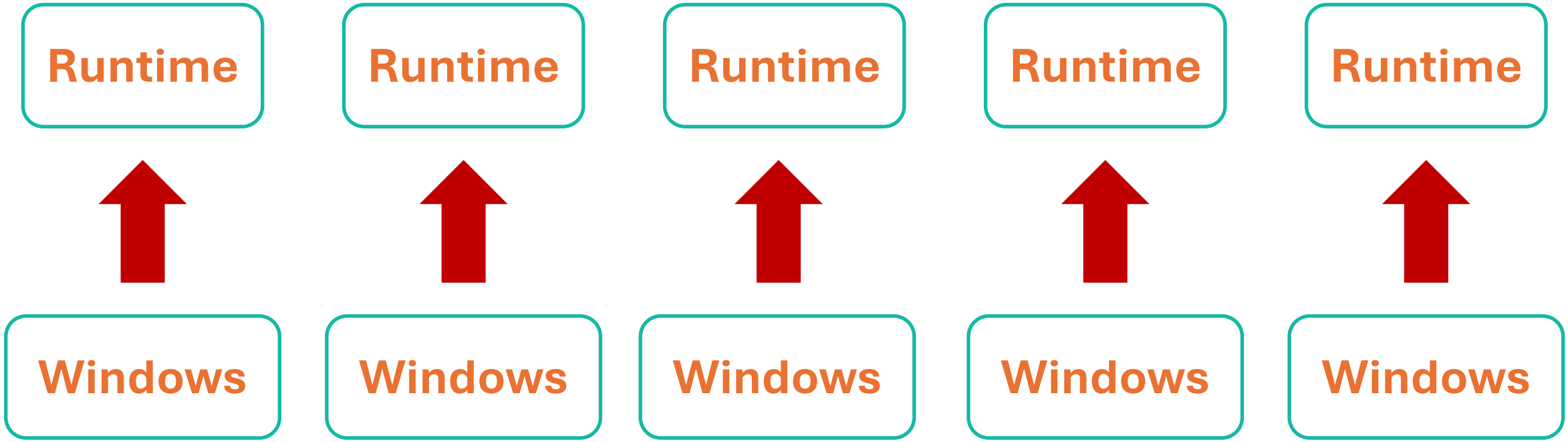
- 하나의 물리 서버에서 여러 독립적인 서비스 실행
- 각 서비스들은 OS도 다르고, 런타임, 라이브러리 등이 모두 다름
- 이 때 하나하나의 가상으로 실행되는 컴퓨터들을 VM이라 부름

Virtual Machine

- 하나의 물리 서버에서 실행되는 여러 독립 서비스들은 서로 다른 OS, 런타임, 라이브러리 등을 사용할 수 있음
- 물리적 하드웨어의 CPU, RAM, SSD 등의 자원을 가상화하여 각 서비스에 할당하고, 서비스를 구동하기 위해 생성되는 가상 컴퓨터를 VM이라 함
- 각 VM은 독립적 OS인 Guest OS를 가지며, 다른 VM과 독립적인 환경에서 실행됨
- 각 VM마다 별도의 GuestOS가 필요하므로 RAM, SSD 등의 추가적인 자원이 필요함

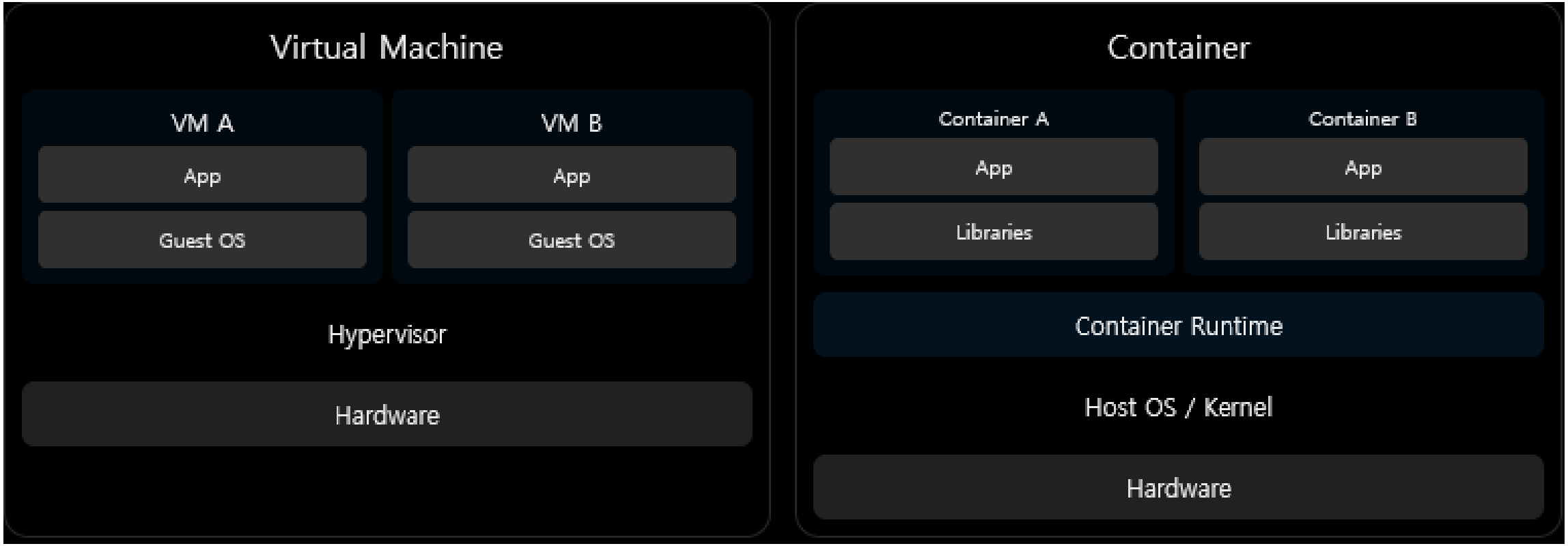


VM의 문제: OS가 겹침



Windows X 5 만큼의 자원이 중복되는 문제가 있음

Container: OS를 호스트의 OS로 고정



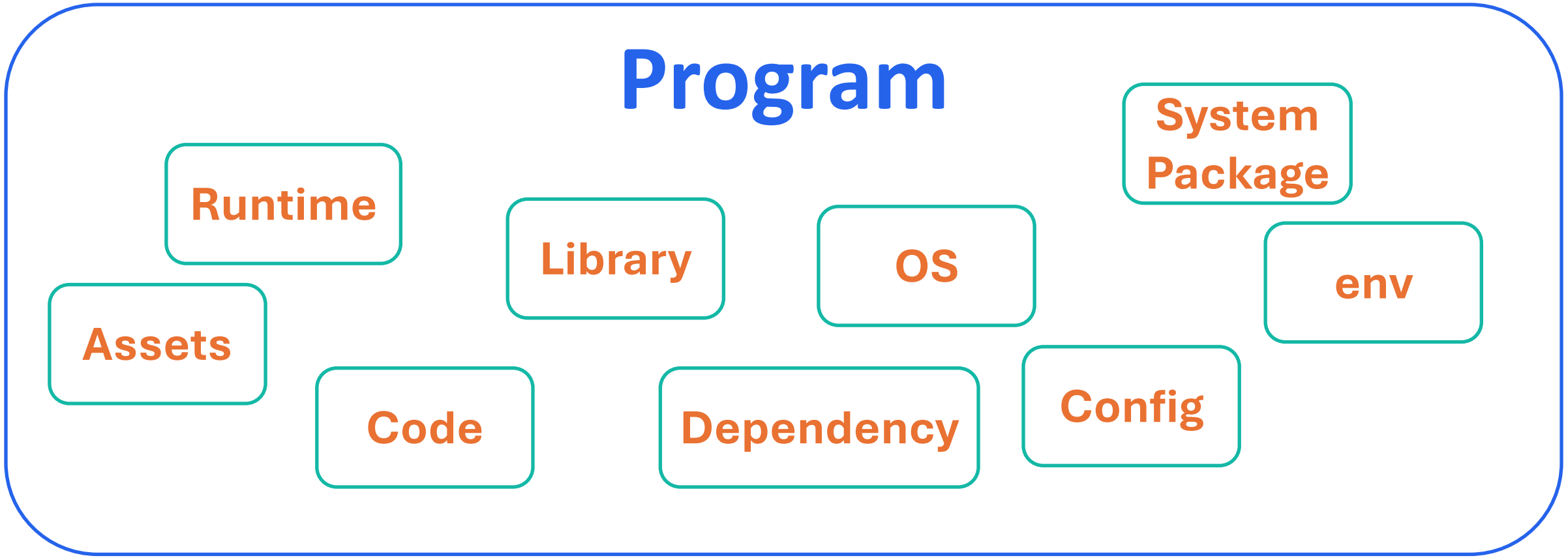
OS가 한번만 실행되어 자원을 아낄 수 있으나, 커널 수준에서
격리되는 VM과 동일한 수준의 격리를 보장하지는 않음

서비스를 배포할때의 문제점





서비스를 배포할때의 문제점



이것들_한번에_배포하기 == 쉽지않음;

서비스를 배포할때의 문제점



컨테이너 만들긴 했는데 이거 생성, 배포, 실행, 관리는 어떻게 함?

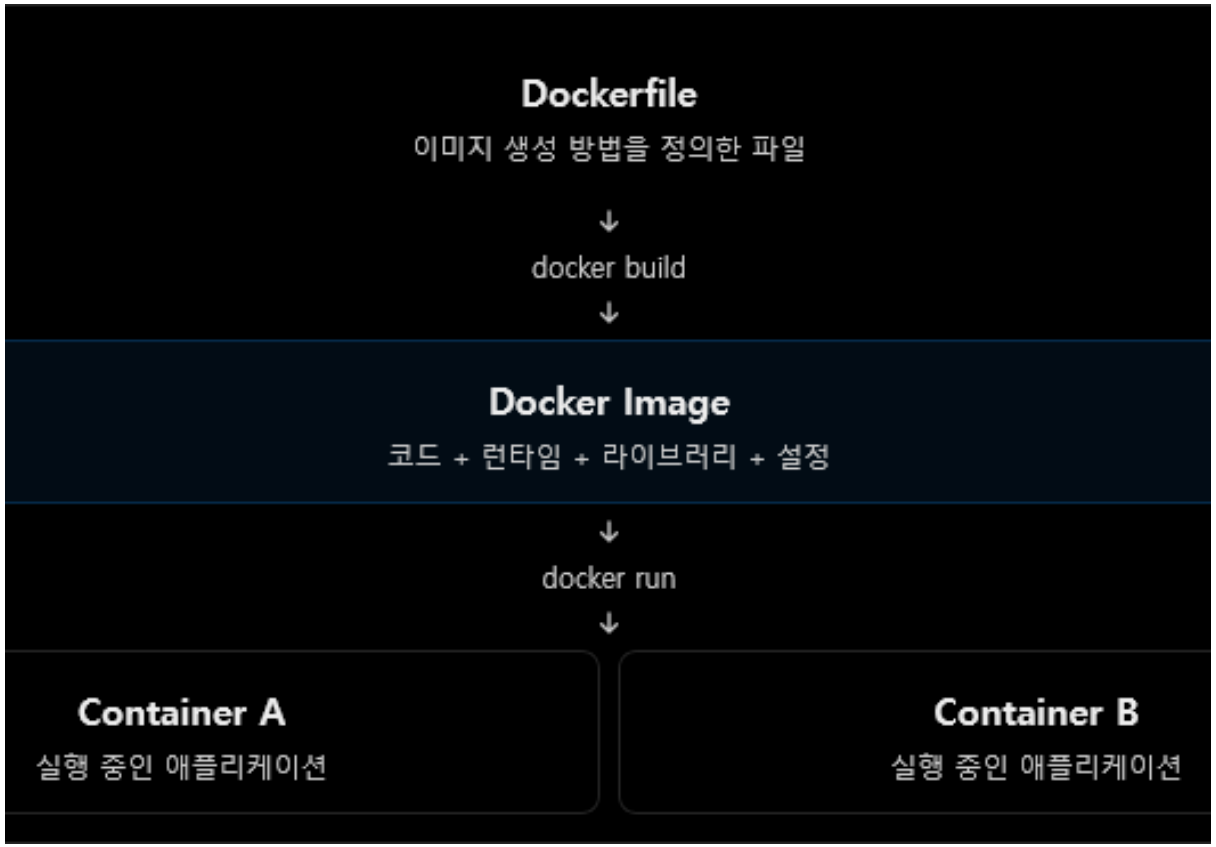
사용자가 많을 때의 문제

이럴 때 Docker라는걸 주로 사용함

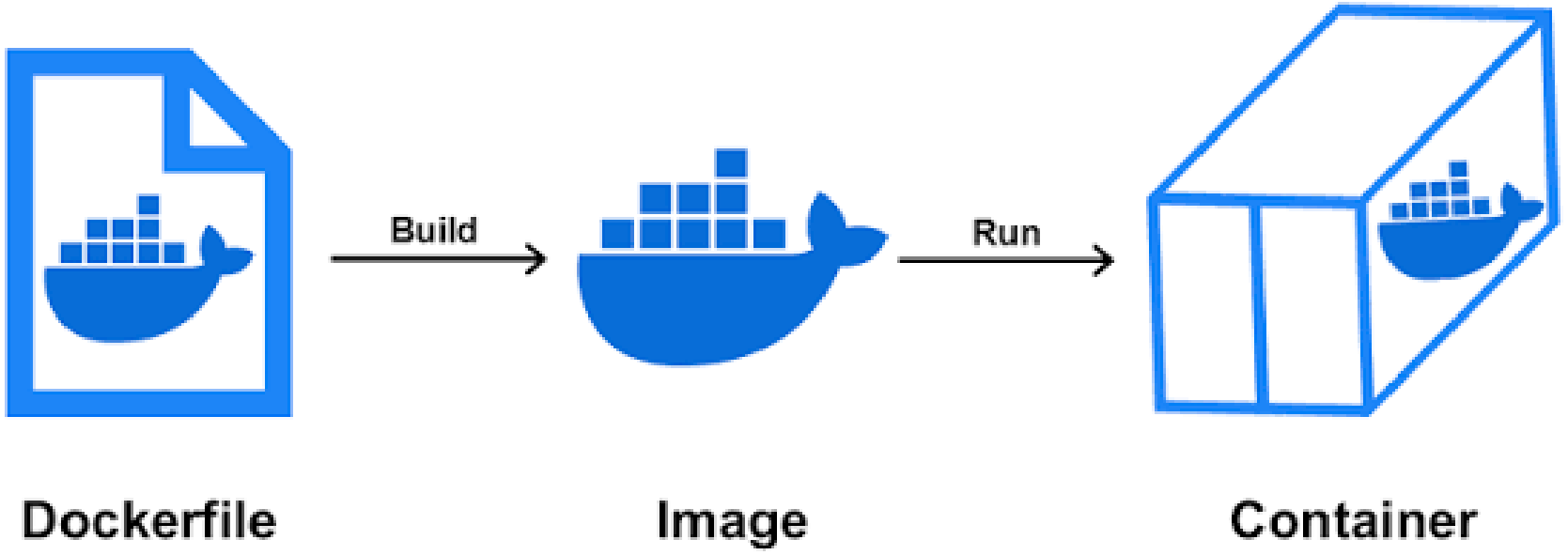


도커의 구조

- Dockerfile: 실행 환경을 어떻게 구성할지 정의
- Build: Dockerfile을 토대로 Docker Image 생성
- Docker Image: 코드, 런타임, 라이브러리 등을 포함한 실행용 패키지
- Run: Docker Image 기반 Container 생성 및 실행
- Container: 이미지를 기반으로 생성된 격리된 애플리케이션 실행 환경

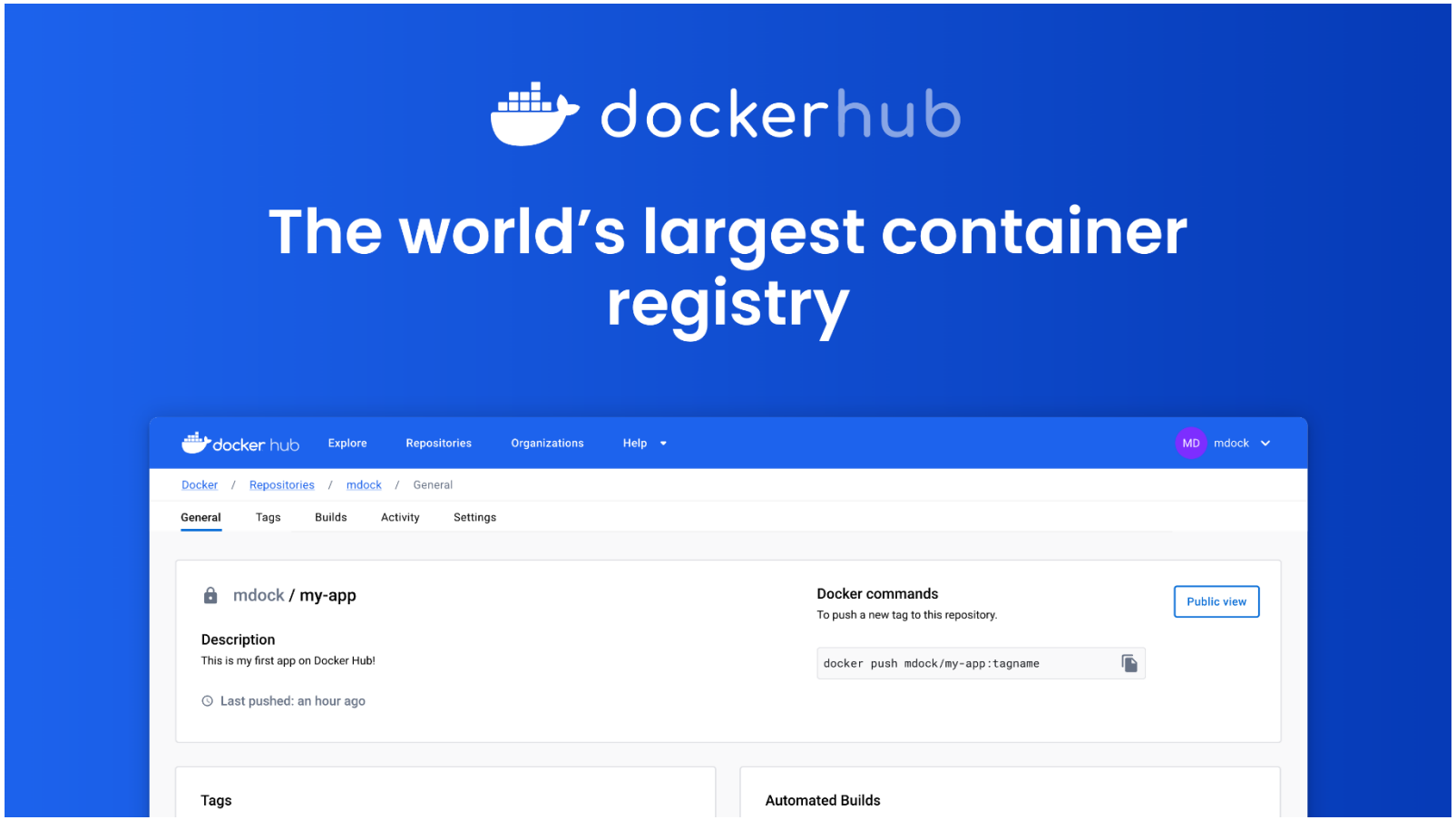


도커로 컨테이너 생성하기



도커 컨테이너를 배포할때는 주로 Docker Image를 배포함

도커로 컨테이너 배포하기



주로 Docker Hub로 배포함

02
클라우드 컴퓨팅

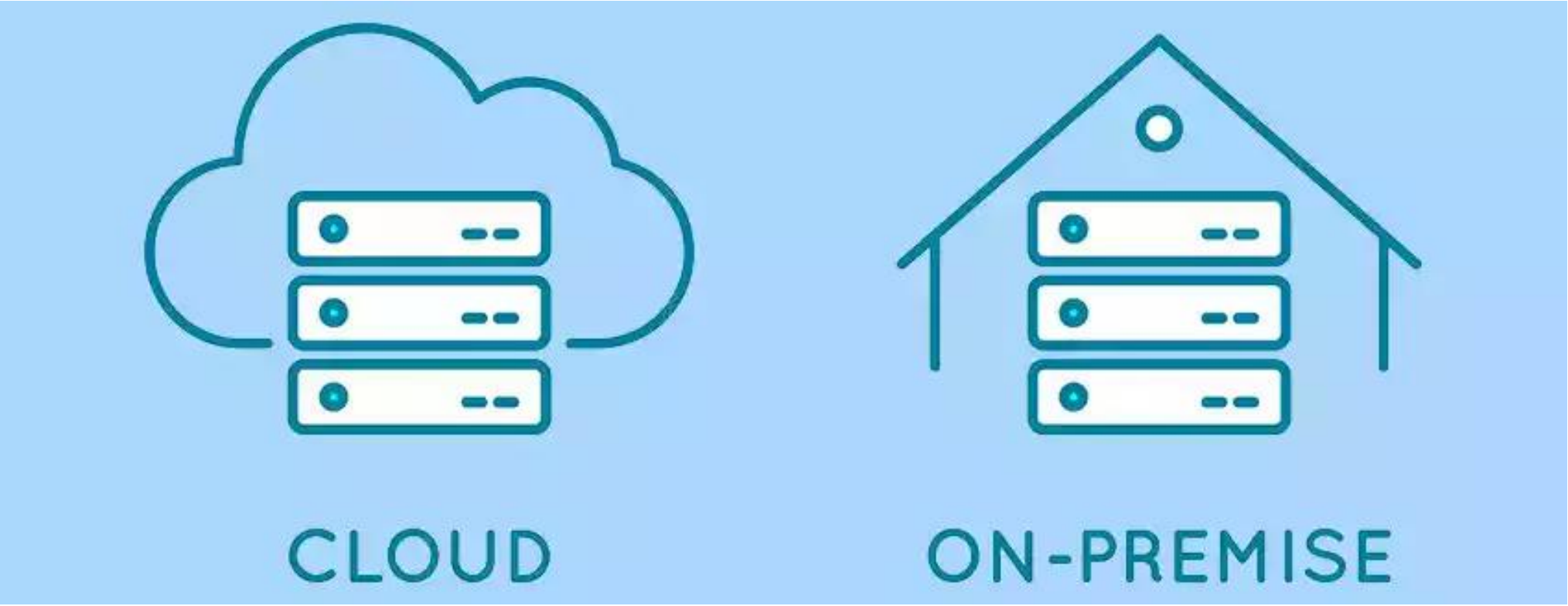
Cloud Computing

클라우드 컴퓨팅



인터넷을 통해 클라우드 서비스 제공자의
컴퓨팅 자원을 공급받는 형태

On-Premise



클라우드 컴퓨팅과는 대조적으로,
자체 서버를 구축하여 운영하는 방식

IaaS & PaaS & SaaS



사용자 관리 영역

서비스 제공자 관리 영역

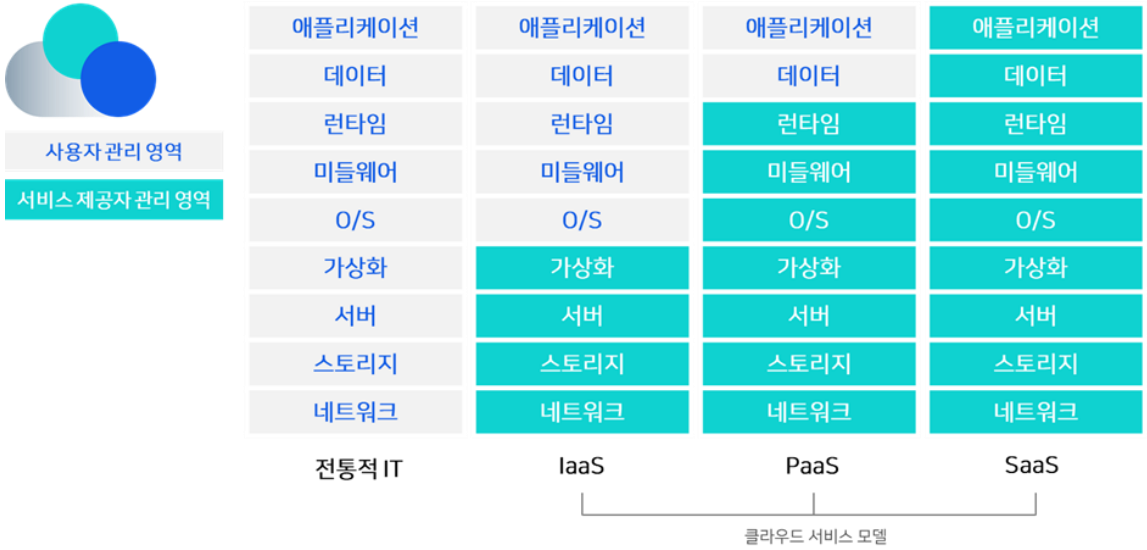
전통적 IT	IaaS	PaaS	SaaS
애플리케이션	애플리케이션	애플리케이션	애플리케이션
데이터	데이터	데이터	데이터
런타임	런타임	런타임	런타임
미들웨어	미들웨어	미들웨어	미들웨어
O/S	O/S	O/S	O/S
가상화	가상화	가상화	가상화
서버	서버	서버	서버
스토리지	스토리지	스토리지	스토리지
네트워크	네트워크	네트워크	네트워크

클라우드 서비스 모델

클라우드 컴퓨팅 서비스를 어디까지 제공할 것인지에 대한 분류

IaaS: Infrastructure as a Service

- 서버의 인프라만 제공하는 서비스
- 프로그램 코드, 실행 OS 등 커스텀 가능



Amazon EC2

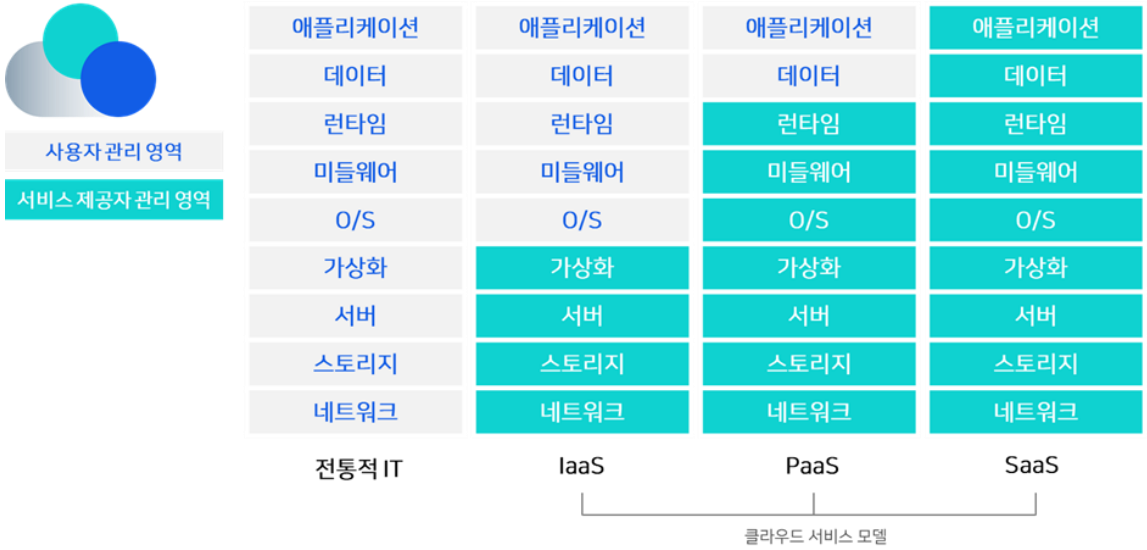
Google Compute Engine

Azure Virtual Machine

Amazon S3

PaaS: Platform as a Service

- 앱의 구축 및 개발 환경 플랫폼까지 서비스로 제공
- 개발자는 애플리케이션 개발과 배포에만 집중할 수 있음



HEROKU

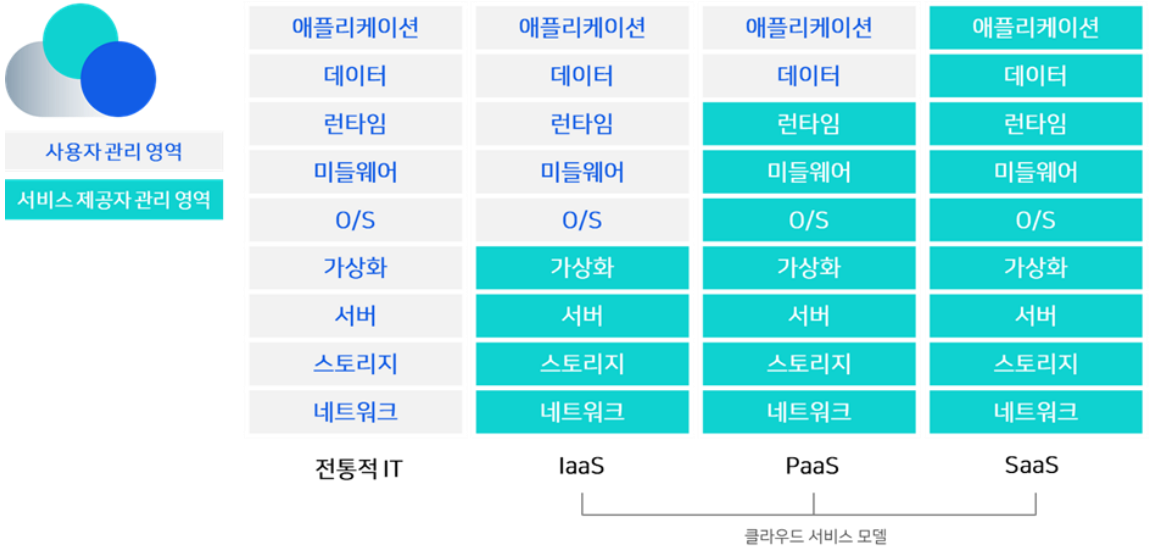
AWS Elastic Beanstalk

Google App Engine

Azure App Service

SaaS: Software as a Service

- 인터넷을 통해 클라우드 기반의 소프트웨어를 제공하는 서비스
- 사용자는 인터넷을 통해 서비스를 즉시 이용 가능





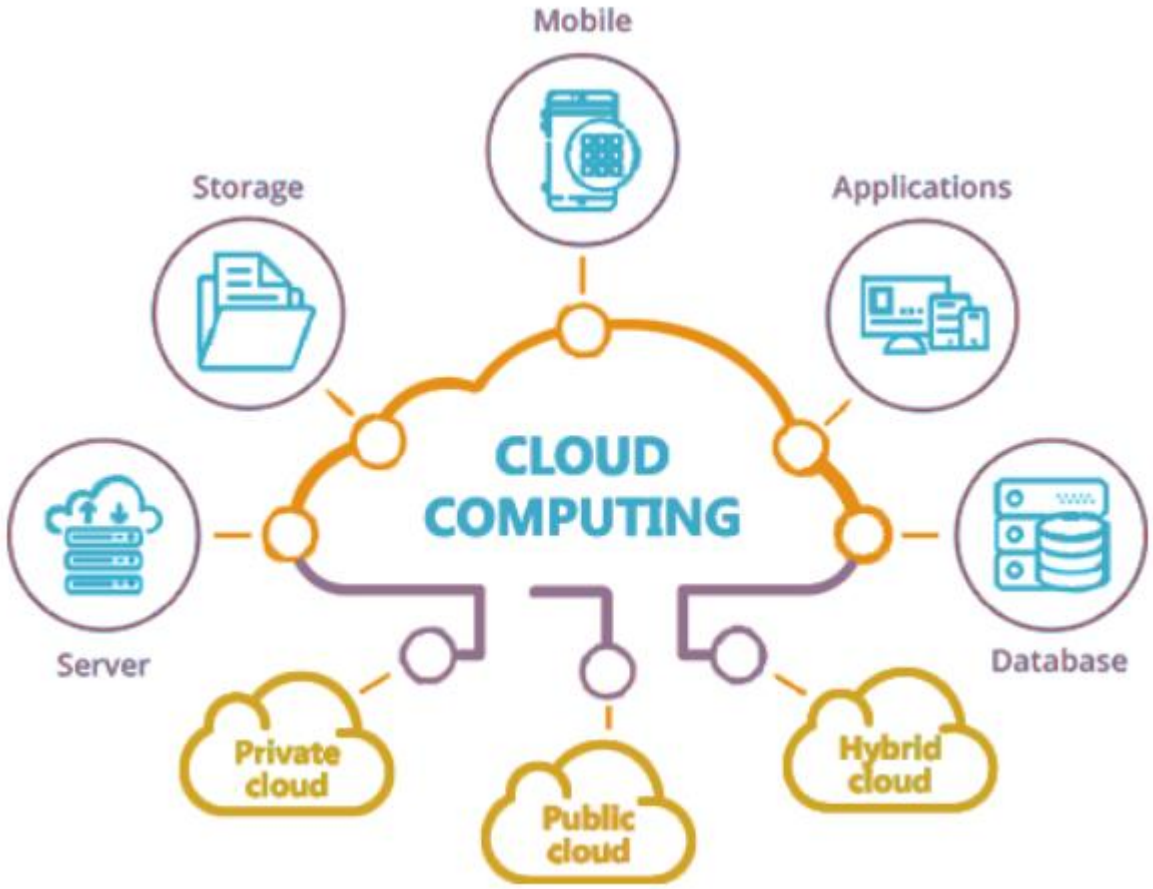






클라우드 컴퓨팅의 장점

- 필요한 컴퓨팅 자원을 즉시 필요한 만큼 생성 및 사용 가능
- 사용자 증가에 따라 서버 자원을 자동으로 확장 가능
- 서버 자원을 사용한 만큼만 비용 지불
- 물리적인 서버의 유지보수를 클라우드 사업자가 담당



03
실무 맛보기

03 실무 맛보기

Refactory

- 협업 과정에서는 필수적으로
  서로가 짠 코드를 리뷰하는
  코드 리뷰 시간이 요구됨

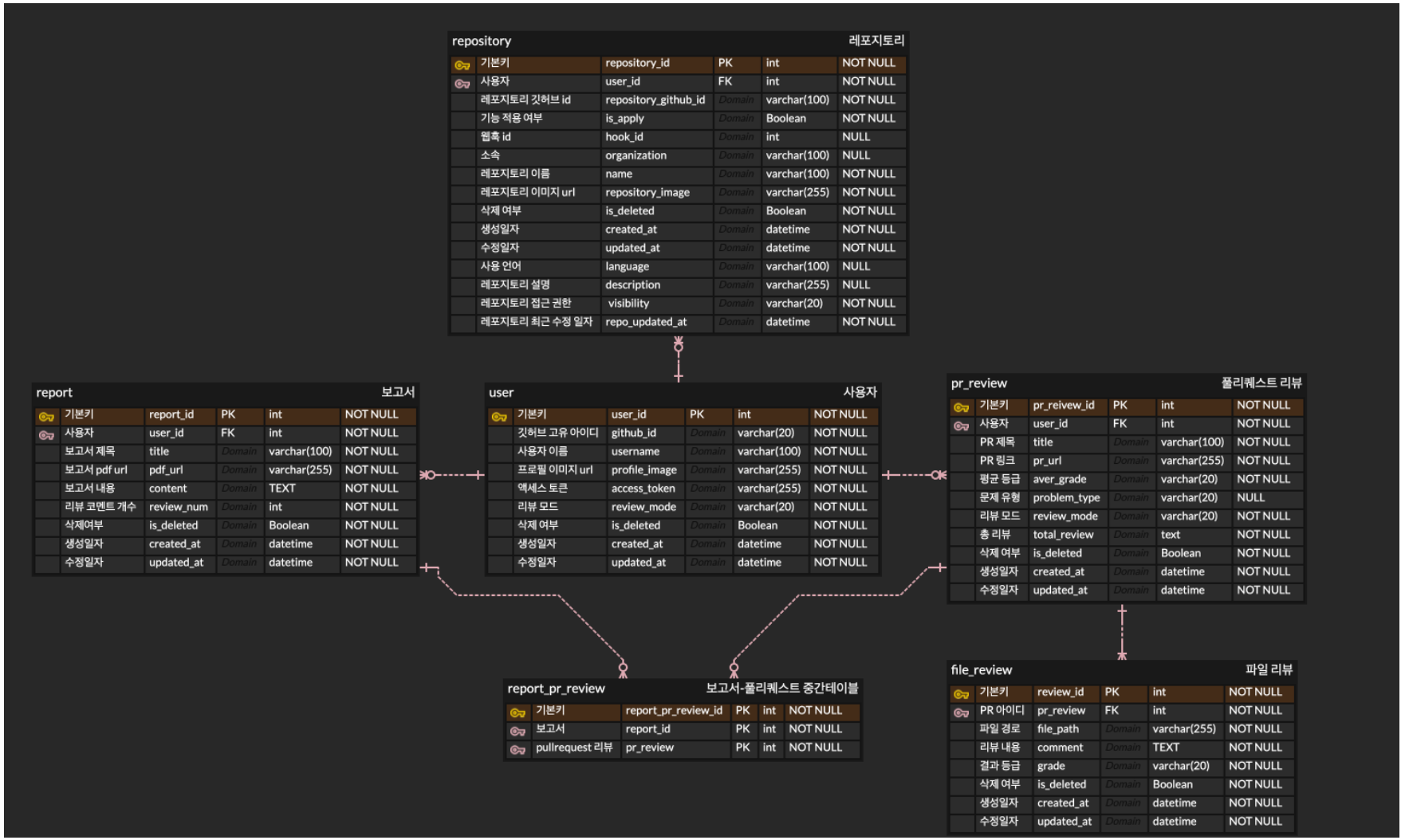
- 기존의 코드 리뷰는 시간도 오래 걸리고,
  기준도 불명확하며, 서로 감정이 상할
  위험도 있었음

- 이를 LLM을 도입하여 시간 단축,
  기준 일관화, 각종 시각화가 추가된
  통합 코드 리뷰 프로그램으로 개발함



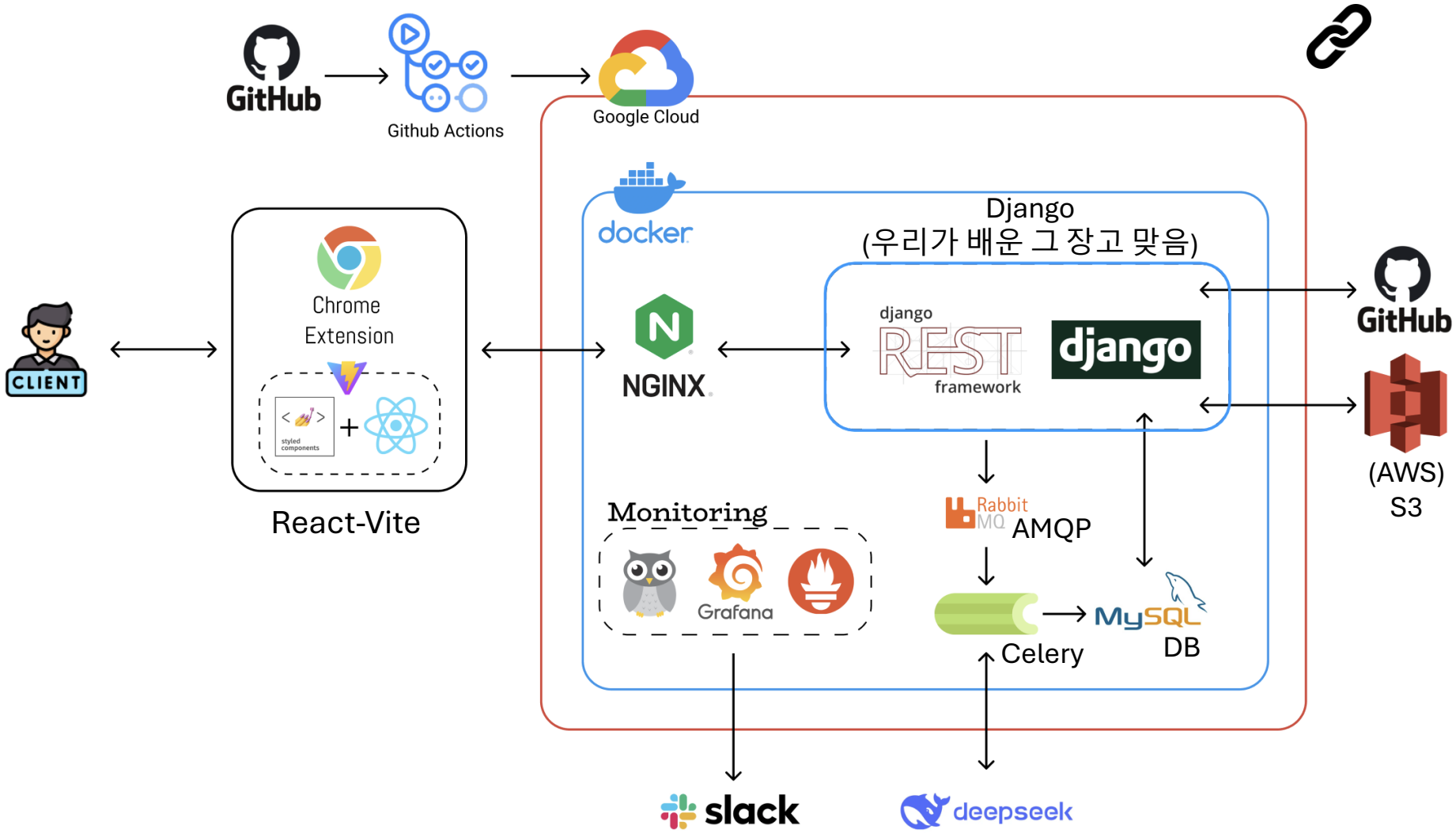
03 실무 맛보기

ERD


repository 레포지토리
기본키 repository_id PK int NOT NULL
사용자 user_id FK int NOT NULL
레포지토리 깃허브 id repository_github_id Domain varchar(100) NOT NULL
기능 적용 여부 is_apply Domain Boolean NOT NULL
웹훅 id hook_id Domain int NULL
소속 organization Domain varchar(100) NULL
레포지토리 이름 name Domain varchar(100) NOT NULL
레포지토리 이미지 url repository_image Domain varchar(255) NOT NULL
삭제 여부 is_deleted Domain Boolean NOT NULL
생성일자 created_at Domain datetime NOT NULL
수정일자 updated_at Domain datetime NOT NULL
사용 언어 language Domain varchar(100) NULL
레포지토리 설명 description Domain varchar(255) NULL
레포지토리 접근 권한 visibility Domain varchar(20) NOT NULL
레포지토리 최근 수정 일자 repo_updated_at Domain datetime NOT NULL
report 보고서
기본키 report_id PK int NOT NULL
사용자 user_id FK int NOT NULL
보고서 제목 title Domain varchar(100) NOT NULL
보고서 pdf url pdf_url Domain varchar(255) NOT NULL
보고서 내용 content Domain TEXT NOT NULL
리뷰 코멘트 개수 review_num Domain int NOT NULL
삭제여부 is_deleted Domain Boolean NOT NULL
생성일자 created_at Domain datetime NOT NULL
수정일자 updated_at Domain datetime NOT NULL
user 사용자
기본키 user_id PK int NOT NULL
깃허브 고유 아이디 github_id Domain varchar(20) NOT NULL
사용자 이름 username Domain varchar(100) NOT NULL
프로필 이미지 url profile_image Domain varchar(255) NOT NULL
액세스 토큰 access_token Domain varchar(255) NOT NULL
리뷰 모드 review_mode Domain varchar(20) NOT NULL
삭제 여부 is_deleted Domain Boolean NOT NULL
생성일자 created_at Domain datetime NOT NULL
수정일자 updated_at Domain datetime NOT NULL
pr_review 풀리퀘스트 리뷰
기본키 pr_reivew_id PK int NOT NULL
사용자 user_id FK int NOT NULL
PR 제목 title Domain varchar(100) NOT NULL
PR 링크 pr_url Domain varchar(255) NOT NULL
평균 등급 aver_grade Domain varchar(20) NOT NULL
문제 유형 problem_type Domain varchar(20) NULL
리뷰 모드 review_mode Domain varchar(20) NOT NULL
총 리뷰 total_review Domain text NOT NULL
삭제 여부 is_deleted Domain Boolean NOT NULL
생성일자 created_at Domain datetime NOT NULL
수정일자 updated_at Domain datetime NOT NULL
report_pr_review 보고서-풀리퀘스트 중간테이블
기본키 report_pr_review_id PK int NOT NULL
보고서 report_id PK int NOT NULL
pullrequest 리뷰 pr_review PK int NOT NULL
file_review 파일 리뷰
기본키 review_id PK int NOT NULL
PR 아이디 pr_review FK int NOT NULL
파일 경로 file_path Domain varchar(255) NOT NULL
리뷰 내용 comment Domain TEXT NOT NULL
결과 등급 grade Domain varchar(20) NOT NULL
삭제 여부 is_deleted Domain Boolean NOT NULL
생성일자 created_at Domain datetime NOT NULL
수정일자 updated_at Domain datetime NOT NULL

03 실무 맛보기

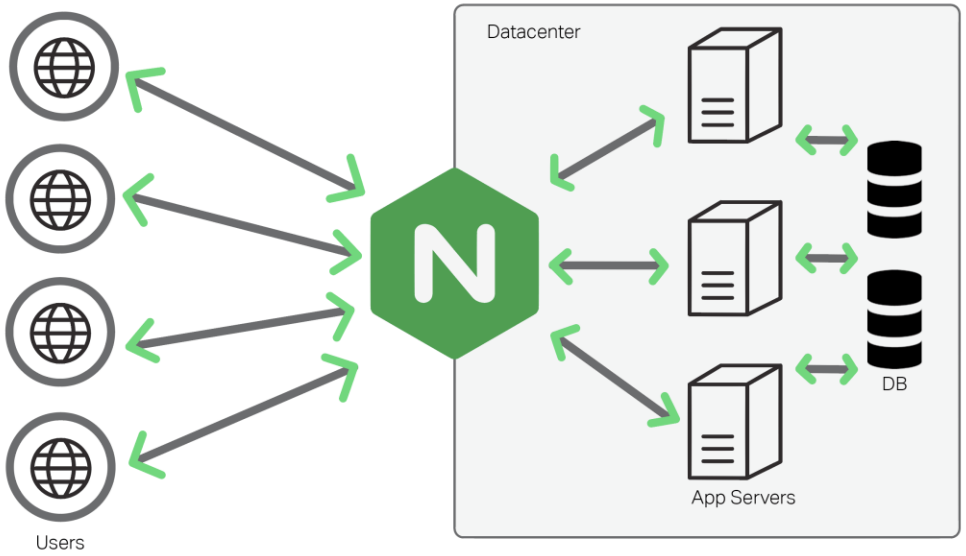
System Architecture



NGINX

- 웹 서버(정적 파일)이자 Reverse Proxy (백엔드 리디렉션) 서버
- 클라이언트의 HTTP 요청을 받아 정적 파일을 제공하거나 적절한 백엔드 서버로 전달
- 로드 밸런서의 역할도 겸하여 여러 클라이언트의 동시 HTTPS 처리 및 로드 밸런싱 지원
- Docker 기반 컨테이너 배포 환경에서 주로 사용됨

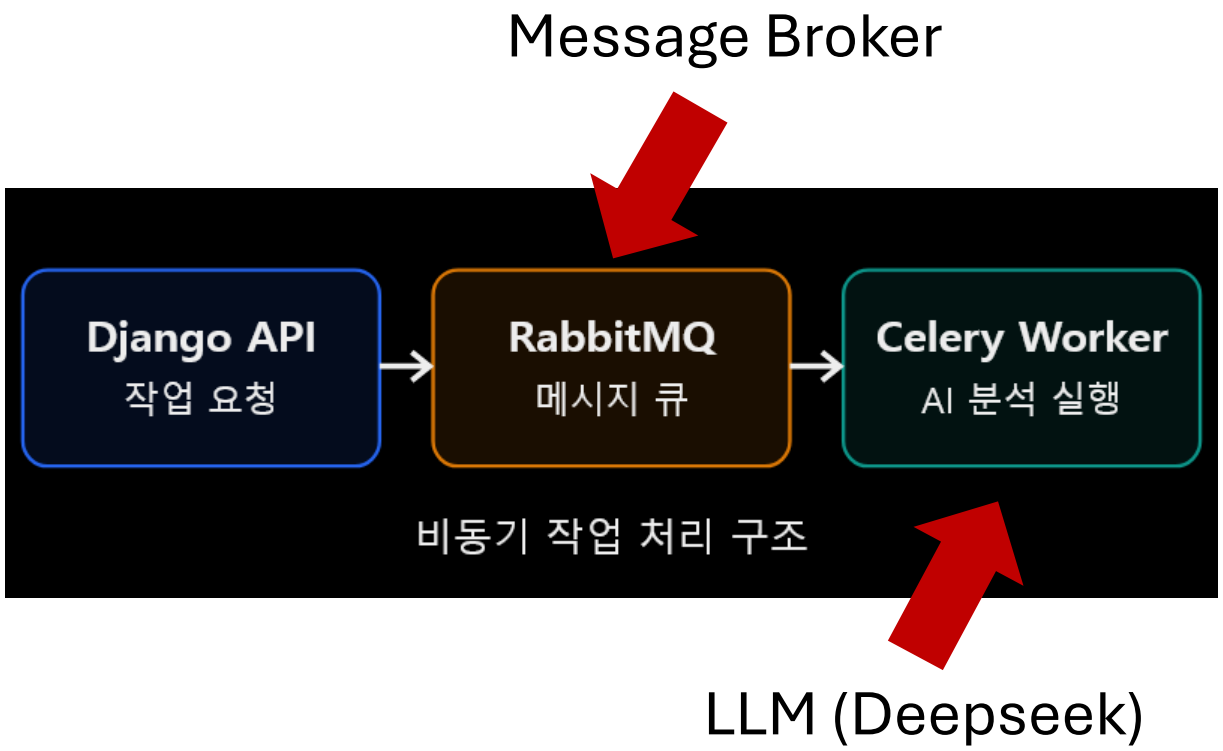




Message Broker

- 서로 다른 서비스 사이에서 메시지를 받아 저장 또는 전달하는 중간 시스템

- 실제 사용 시 사용자가 Django를 통해 LLM에게 요청을 하면 Django는 LLM의 분석이 끝날 때까지 대기할 필요가 없음 (== 비동기 작업 처리 구조)



MQTT & AMQP

- MQTT: Message Queuing Telemetry Transport

  주로 가벼운 메시지의 송수신에 쓰이며,
  IoT, 센서 등에서 쓰임
  메시지 브로커로는 주로 Mosquitto가 쓰임

- AMQP: Advanced Message Queuing Protocol

  신뢰성 있는 메시지 전달 및 라우팅에 주로
  쓰이며, 백엔드, 작업 큐 등에서 쓰임
  메시지 브로커로는 주로 RabbitMQ가 쓰임









Apache Kafka

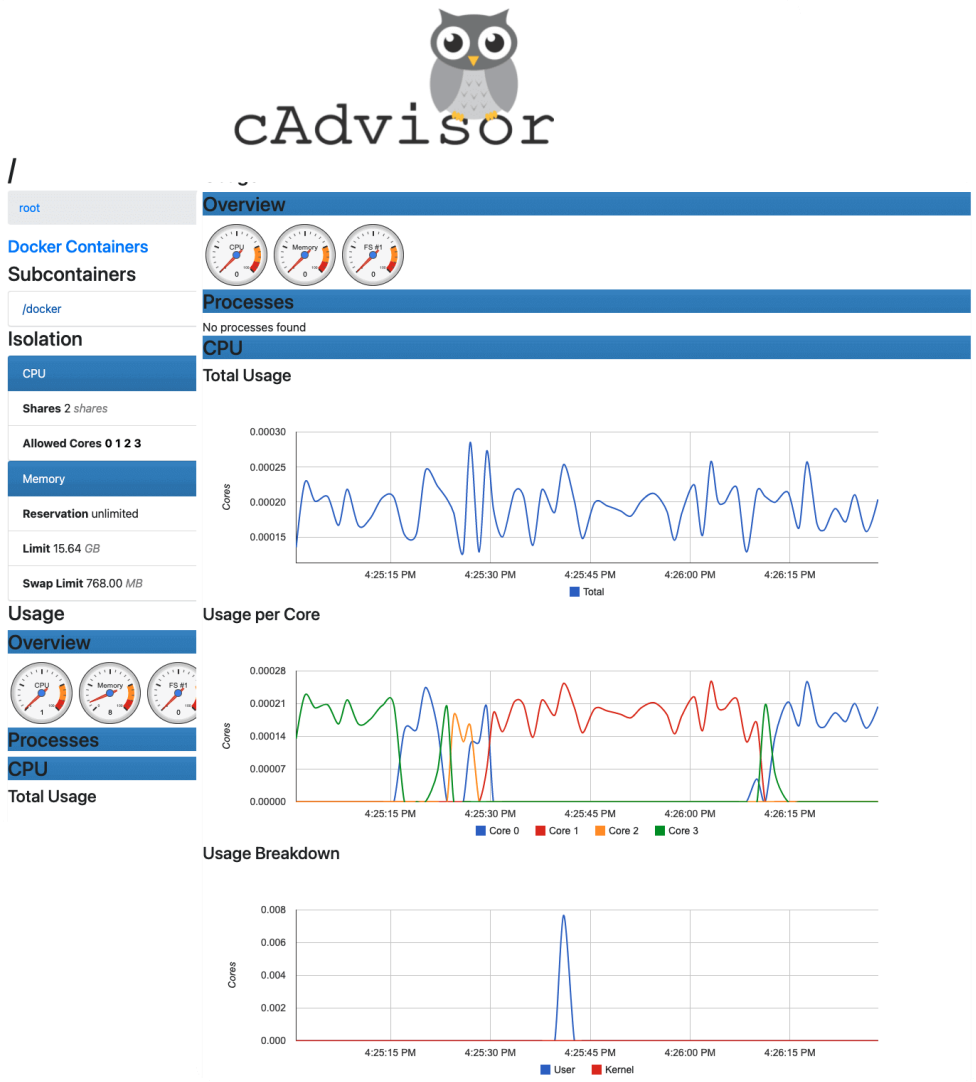
- 대용량 데이터를 실시간으로 수집, 저장, 전달하는 분산 이벤트 스트리밍 플랫폼
- 일반적인 메시지 브로커와 달리 메시지를 소비한 후에도 일정 시간동안 데이터를 유지하여 재사용성을 높임
- LinkedIn같은 거대한 서비스에서 주로 사용됨



Monitoring: cAdvisor

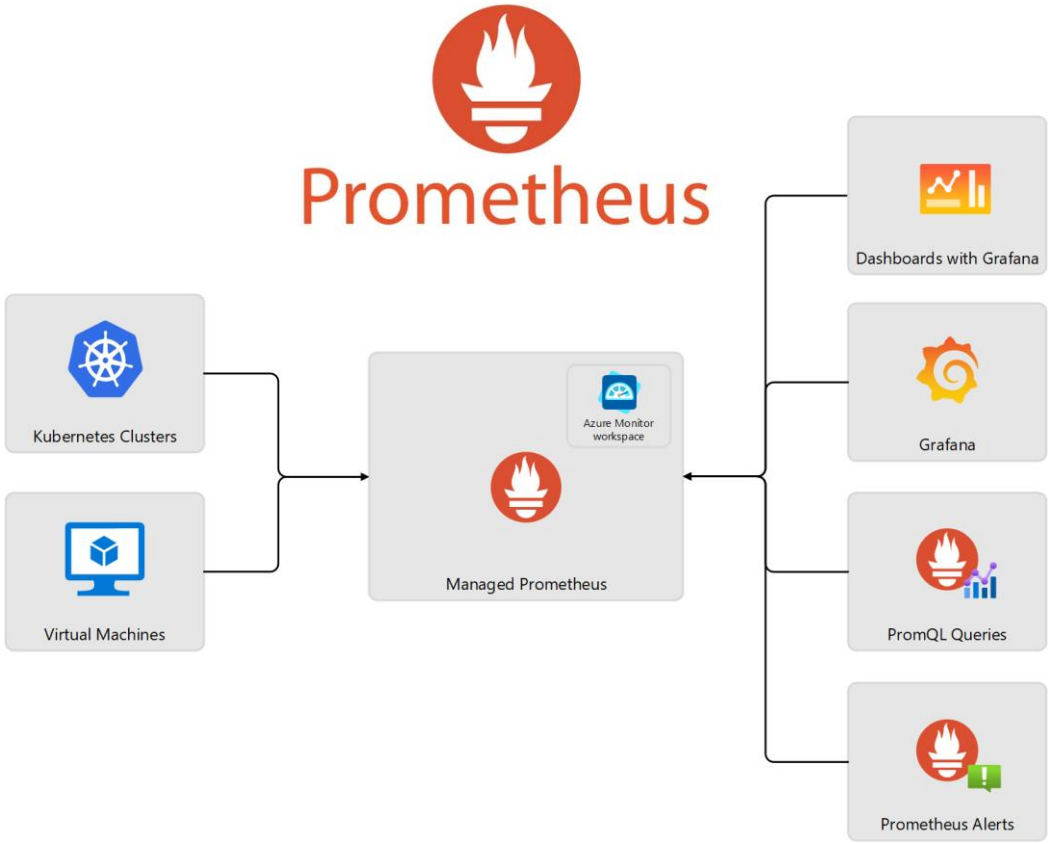
- 실행되는 컨테이너가 소모하는 자원을 모니터링하는 도구

- CPU 사용률, 메모리 사용량, 네트워크 트래픽, 컨테이너 상태 등을 모니터링함



Monitoring: Prometheus

- cAdvisor 등에서 수집되는 정보들을 주기적으로 수집(Pull)하고 저장
- SQL과 비슷한 PromQL을 통해 저장된 데이터에 대한 조회 및 분석 지원
- 한 순간의 정보를 주로 저장하는 cAdvisor와 달리 시계열 정보를 주로 저장함



03 실무 맛보기

Monitoring: Grafana

- cAdvisor, Prometheus 등 다양한 데이터 소스에 연결 가능

- CPU, 메모리 사용량과 같은 여러 요소들을 그래프 및 대시보드로 시각화하고 이상상황 발생 시 알림 설정 지원

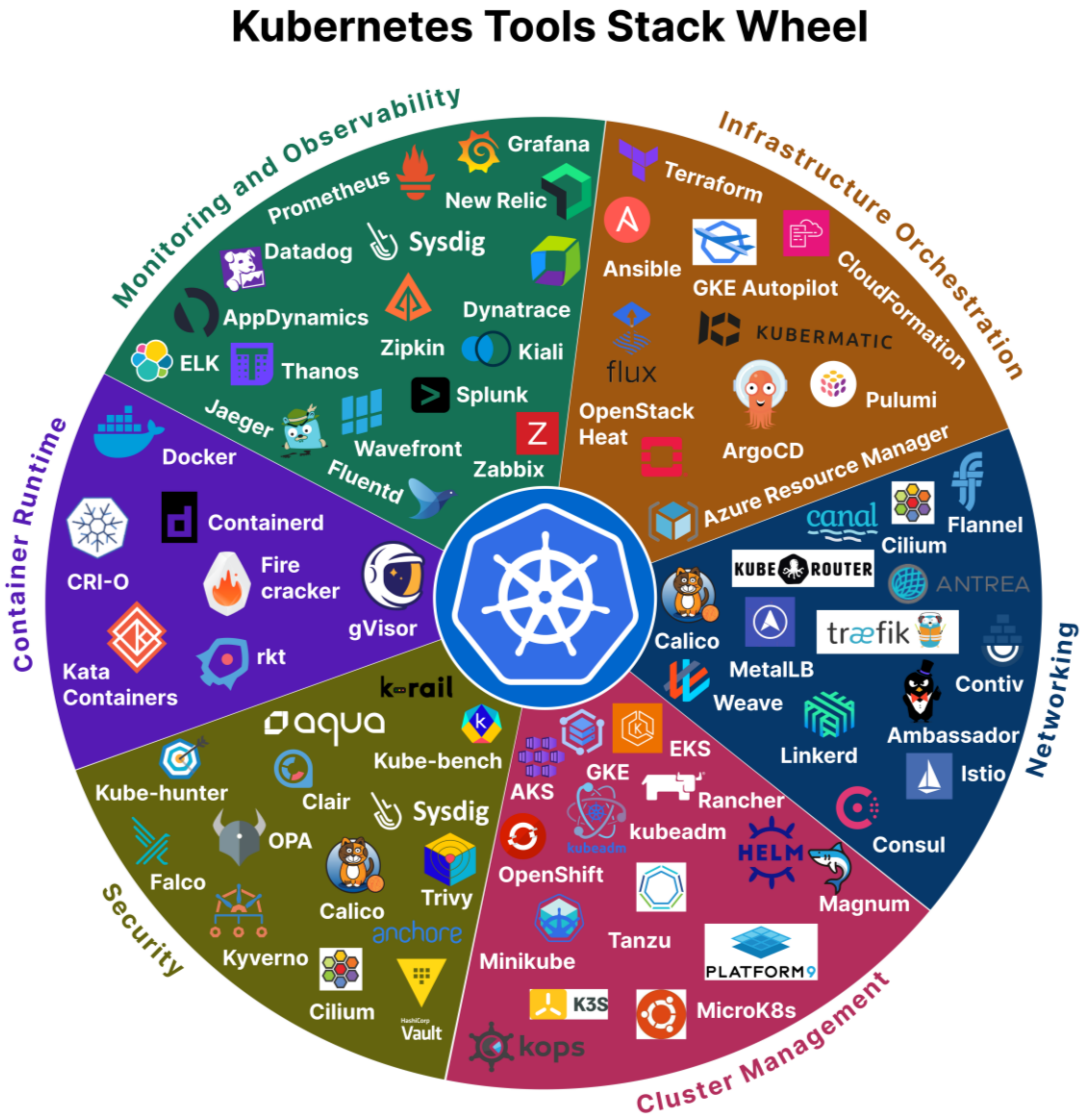


Kubernetes

- 여러 서버에 분산된 수많은 컨테이너들을 자동으로 배포하고 관리하는 컨테이너 오케스트레이션 플랫폼

- 기본적으로 컨테이너를 묶은 Pod라는 단위로 관리되며, Pod들은 여러 서버에 적절하게 분산하여 배치됨

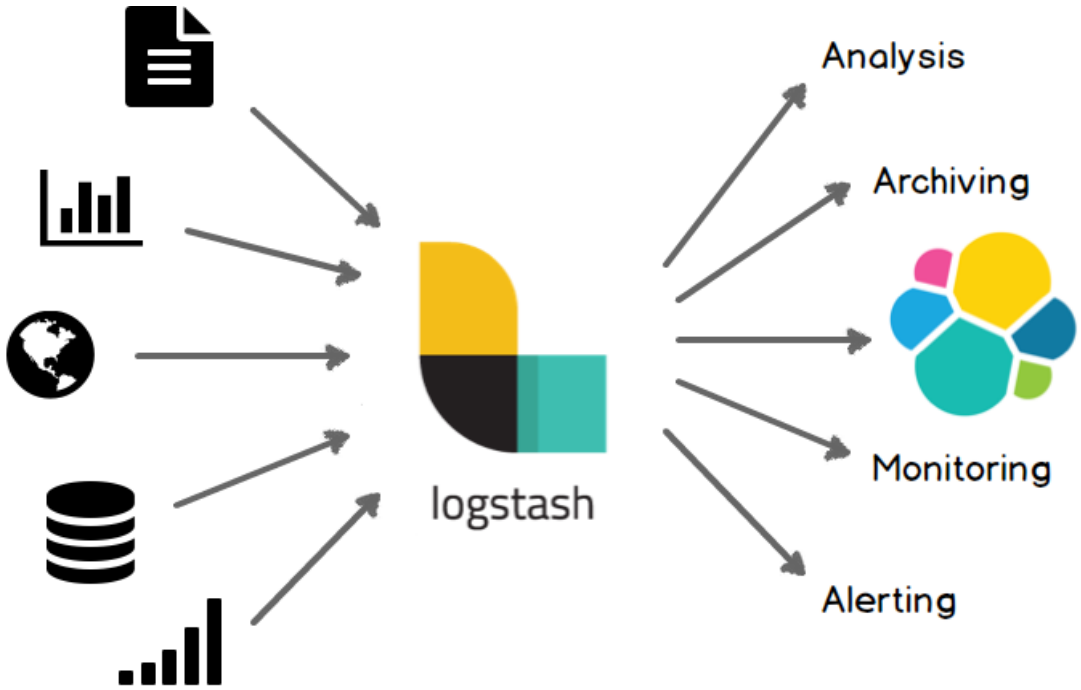
- 컨테이너에 장애가 발생하면 자동으로 복구하고 트래픽 증가에 따라 Pod 수를 유동적으로 조정



ELK: Logstash

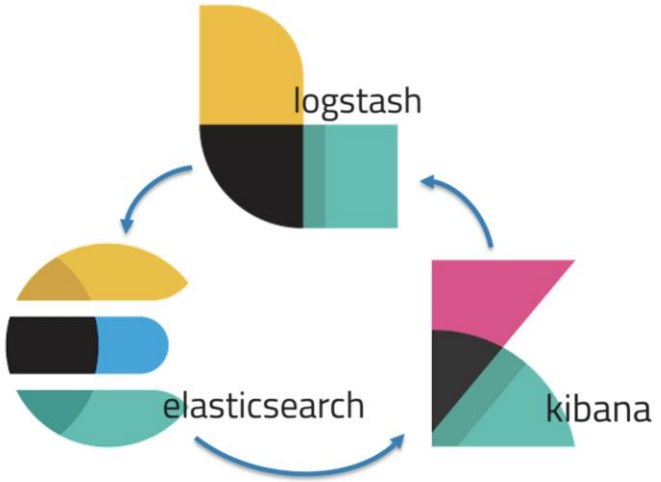
- 여러 서버, 여러 애플리케이션에서 발생하는 수많은 로그를 수집하고 필요한 형태로 가공하는 데이터 처리 파이프라인

- 가공된 로그들은 Elastic search로 전달됨



ELK: Elasticsearch

- Logstash로부터 전달받은 로그 데이터를 저장하고 빠르게 검색 및 분석할 수 있도록 지원하는 검색 및 분석 엔진





ELK: Kibana

- Elasticsearch에 저장된 데이터를
  검색 및 분석하고, 그래프와 대시보드 등의
  형태로 시각화하는 도구

- Monitoring 도구들과 ELK를 조합하면
  Grafana로 이상 발생 시점을 파악하고,
  그 시점의 로그를 Kibana로 조사하는 등의
  고차원적인 시스템 유지보수가 가능해짐



03 실무 맛보기

CI/CD

- CI: Continuous Integration

  여러 개발자가 작성한 코드를
  공통 저장소에 통합하고, 빌드 및 테스트를
  자동으로 수행하는 과정

- CD: Continuous Delivery/Deployment

  실제 배포는 수동 승인 / 테스트 통과하면 배포까지 자동화

  CI를 통과한 코드를 실제 서비스 환경에
  배포할 수 있도록 준비 또는 배포를
  자동화하는 과정





DevOps

- Development + Operations의 합성어
  개발팀과 운영팀의 협업을 통해
  소프트웨어의 개발, 배포, 운영 과정을
  효율화 하는 방법론

- CI/CD를 통해 코드 빌드, 테스트,
  배포 과정 등을 자동화하여 개발 속도 및
  배포 안정성 향상

- Docker, Kubernetes 등을 활용하여
  실행 환경 표준화 및 배포, 확장 자동화

- Grafana, ELK 등의 도구를 활용하여
  서비스 상태를 관찰하고 장애에 신속히 대응

